Studio Nx

AI 구현 설명 38 — P1-07 가입 · 로그인 JS

2026-10-07

AI 구현 설명 38 — P1-07 가입 · 로그인 JS (폼 검증 · 중복 확인 · 로그인 모달 · 헤더 전환)

항목	내용
작성	Claude (AI) · 2026-10-07
작업	P1-07 가입 · 로그인 JS — PRD 01 FR-101 · FR-102 · FR-103 · FR-108
브랜치	`feature/phase1/p1-07-auth-js`
바꾼 파일	`static/js/core/auth-ui.js` · `static/js/pages/home.js` · `static/js/pages/join.js` · `static/css/auth.css` (한 줄)
진행 방식	**함수 하나씩** 구현 → 사용자 검토 → 다음. 실제 코드는 **전통 문법**(`?.` · `??` 없음), 최신 문법 버전은 함수 문서의 "최신 문법으로 쓰면" 에
함수별 설명	`docs/ai-notes/javascript/P1-07 - 가입·로그인 JS/09 ~ 17` (md · PDF), 합본 P1-07_auth_js.pdf
실제 사이트 시험	39_p1-07_site_test.md — 사람처럼 클릭 · 타이핑, 11개 모두 통과
이전 문서	37_p1-04_05_site_test.md

1. 한 일

P1-04 · P1-05 서버 API 와 P1-02 공통 JS 를 **화면의 폼 · 버튼 · 모달에 연결**했다. 이제 사람이 직접 가입하고 로그인할 수 있다.

화면	이제 되는 것
`join.html`	이메일 · 닉네임 실시간 중복 확인, 비밀번호 조건 ✓, 확인 칸 비교, 가입 전송 → 홈의 로그인 모달
로그인 모달 (`home.html` · `join.html`)	로그인 → 헤더 전환 또는 `next` 페이지로. 실패 이유는 숨김
헤더 (모든 페이지)	로그인 상태면 My Page · My Projects, 아니면 Sign In · Join Us
`home.html#login?next=…`	로그인 모달 자동 열림
Google · Discord · SoundCloud 버튼	"곧 지원" 안내 한 줄 (연동은 나중)

2. 함수 9개 (번호는 P1-02 의 01 ~ 08 에 이어서)

#	파일	함수	하는 일	설명
09	`auth-ui.js`	`setFieldState(field, state, message)`	칸 하나의 상태(정상 · 오류 · 통과)와 아래 문구	09
10	`auth-ui.js`	`initPasswordToggles(root)`	비밀번호 Show / Hide	10
11	`auth-ui.js`	`initOAuthPlaceholders(root)`	소셜 로그인 버튼 → "곧 지원" 안내	11

#	파일	함수	하는 일	설명
12	`auth-ui.js`	`initAuthState()`	로그인 상태 확인 → 헤더 버튼 전환 (`<html data-auth>`)	12
13	`auth-ui.js`	`initLoginForm(options)`	로그인 모달 전송 → 토큰 저장 → 헤더 전환 / `next`	13
14	`home.js`	`initLoginModalFromHash()`	`#login…` 이면 모달 자동 열기, 닫으면 주소 정리	14
15	`join.js`	`initDuplicateChecks()`	이메일 · 닉네임 중복 확인 (Check 버튼 · 입력 멈추고 0.5초)	15
16	`join.js`	`initPasswordRules()`	비밀번호 조건 ✓ · 허용 안 되는 글자 · 확인 칸 비교	16
17	`join.js`	`initJoinForm()`	가입 전송 — 전체 검사 → 서버 → 홈의 로그인 모달	17

3. 처음 가입하는 사람의 흐름

순서	화면	일어나는 일	함수
①	join.html	이메일 · 닉네임을 치고 멈추면 0.5초 뒤 `GET /auth/check-…` → ✓ Available	15 → 09
②	join.html	비밀번호를 칠 때마다 조건 ✓, 확인 칸이 다르면 알림, Show 로 보기	16 · 10
③	join.html	Create account → 모든 칸 검사 → `POST /auth/signup` → `home.html#login`	17
④	home.html	로그인 모달 자동 열림	14
⑤	로그인 모달	`POST /auth/login` → Access Token 은 메모리, Refresh 는 HttpOnly 쿠키	13
⑥	모든 페이지	헤더가 My Page · My Projects 로. 새로 고침해도 쿠키로 유지	12

로그인이 필요한 페이지에서 온 경우: `requireLogin()` (P1-02 07)이 `home.html#login?next=/myPage.html` 로 보냄 → 14 가 모달을 엶 → 13 이 로그인 후 `next` 로 이동.

4. 함께 바꾼 것

파일	내용	이유
`auth.css`	`.oauth-note` 한 줄 (가운데 정렬 · 위 여백)	11 의 안내 줄
`join.js` 아래쪽	`initLoginForm({ onSuccess: (next) => { location.href = next || 'home.html'; } })`	가입 화면에서 로그인하면 가입 화면에 남을 이유가 없다
`home.js` import	`parseLoginHash` 추가	14
`join.js` import	`api` 추가	15 · 17

5. 템플릿과 달라진 점

항목	템플릿	구현	이유
12 `initAuthState`	일반 함수	`async` + `true/false` 돌려줌	로그인 확인이 끝나야 다음 일을 하는 페이지가 기다릴 수 있다

항목	템플릿	구현	이유
13 `onSuccess`	페이지마다 넘김	없으면 기본 동작(모달 닫고 헤더 전환)	home.html 은 넘기지 않아도 된다
14 `next`	따로 저장	저장 안 함 — 로그인 순간 주소에서 읽음	저장 장소를 하나 줄임
14 `hashchange`	없음	받음	홈을 보는 중에 `#login` 링크를 눌러도 열림
11 안내	`alert` 또는 안내	화면 안 안내 한 줄	하던 일을 막지 않는다
17 가입 후	자동 로그인 또는 로그인 모달	로그인 모달	서버 설계(가입 · 로그인 분리)와 맞춤, "Keep me signed in" 을 직접 고름

6. 시험

방법	대상	결과
헤드리스 Chrome 으로 실제 페이지를 열고 그 안에서 함수 시험 (실제 서버 · MySQL)	09 ~ 17	11 · 9 · 8 · 9 · 12 · 8 · 15 · 17 · 10 = **99개 통과**
사람처럼 클릭 · 타이핑하는 실제 사이트 시험	가입 → 로그인 → 이동 → 로그아웃 → `next`	**11개 통과** (39)

시험으로 찾아 고친 빈틈 4개

함수	빈틈	고친 것
15	통과 뒤 값이 바뀌어 "사용 중" 이 나와도 옛 통과 표시(`data-checked`)가 남음	확인할 때마다 먼저 지움
16	확인 칸까지 맞게 친 뒤 비밀번호를 고쳐도 "맞음" 으로 남음	비밀번호를 고치면 확인 칸도 다시 비교
17	동의를 체크해도 빨간 동의 안내가 다음 전송까지 남음 (사이트 시험에서 발견)	체크하면 바로 지움
(시험 도구)	페이지 스크립트가 끝나기 전에 시험 시작	2.5초 기다림

7. 남은 것 · 다음 작업이 쓰는 곳

작업	쓰는 것
P1-10 회원 정보 화면 JS	09 `setFieldState` · 10 · 16 과 같은 방식의 비밀번호 칸
P1-14 내 프로젝트 화면 JS · P1-17 마이페이지 JS	`requireLogin()` 연결, **Log out 버튼**(`initLogoutButton`, 지금은 템플릿)
소셜 로그인 (나중)	11 의 안내 부분을 연동 주소로 바꾸면 됨